

# ELI — Project Overview

ELI v2.1.66

August 2026

## Contents

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>ELI MKXI — Full Project Breakdown &amp; Assessment</b>  | <b>1</b> |
| 1. What ELI is . . . . .                                   | 1        |
| 2. Scale & shape . . . . .                                 | 2        |
| 3. Architecture, layer by layer . . . . .                  | 2        |
| 4. What’s genuinely strong . . . . .                       | 3        |
| 5. Where it’s weak (grounded in the sweep) . . . . .       | 3        |
| 6. Honest verdict on “frontier, ground-breaking” . . . . . | 4        |
| 7. Highest-leverage work (the next month) . . . . .        | 4        |

## ELI MKXI — Full Project Breakdown & Assessment

**Updated for v2.1.51 (August 2026).** The primary install path is now the prebuilt, CI-launch-tested installers on GitHub Releases (Windows Setup.exe, macOS dmg, Linux AppImage) with first-boot GPU (CUDA/Vulkan/Metal) and starter-model offers; data lives in a per-user ELI\_v2 folder that survives upgrades. Source installs below remain fully supported.

A grounded, total-project read: what ELI is, its scale and shape, the architecture layer by layer, what’s strong, where it’s weak (with numbers), an honest verdict, and the highest-leverage work. Companion to orchestration\_and\_agents.md (agent/bus detail).

Method note: this is built from deep reads of the core modules (engine, executor, router, agent\_bus, orchestrator, planners, memory, grounding gate, vision, identity, security, reasoning modes) plus a complete structural and code-health sweep (LOC distribution, debt markers, duplication, repo hygiene). It is grounded in a deep read of every package this session — claims are grounded in what was actually inspected.

---

### 1. What ELI is

A **local-first, model-agnostic personal AI** — no cloud, no telemetry, offline-by-default and hard-gated at the socket boundary (internet is an owner-controlled, monitored opt-in, not a runtime dependency; one-time opt-in HuggingFace model downloads are the only network event unless the owner enables the Internet toggle). It bundles: GGUF inference (llama-cpp), a PySide6 desktop GUI, persistent SQLite+FTS5+FAISS memory, a knowledge graph, a 12-stage cognitive pipeline with a parallel multi-agent bus, a deterministic grounding/evidence layer to fight confabulation, local vision (Qwen2.5-VL hot-swap + Moondream co-resident), TTS/STT, OS control, a plugin system, a proactive daemon, and a LoRA self-training loop.

**As of 2026-06-28 it has a second front end:** a self-hosted **FastAPI web app + dashboard PWA** (api/server.py) — chat, a live telemetry dashboard, **ELI’s own MQTT smart-home** (rooms, scenes, real

automations; Home Assistant removed), multi-user **RBAC** (admin/member/viewer), a **tamper-evident hash-chained audit trail**, collaborative **research corpora**, **browser voice** (local whisper STT + Piper TTS), and the monitored Internet toggle — launchable in-process from the desktop GUI. It is a cognitive operating system for one machine (and, optionally, the people the owner shares it with), not a chat wrapper.

## 2. Scale & shape

**143,432 LOC across 392 Python files** (eli/), plus the FastAPI web server (api/server.py, ~3,884 lines with an embedded dashboard PWA) and 205 test files. (*measured 2026-06-28.*)

| Subsystem                                    | LOC   | Files | Role                                                     |
|----------------------------------------------|-------|-------|----------------------------------------------------------|
| execution/                                   | 23.1k | 14    | router + executor<br>(action dispatch)                   |
| runtime/                                     | 22.6k | 80    | grounding, evidence,<br>introspection, surfaces          |
| gui/                                         | 20.3k | 20    | PySide6 desktop app                                      |
| kernel/                                      | 14.5k | 8     | the engine (pipeline<br>driver)                          |
| cognition/                                   | 14.0k | 27    | agent bus, orchestrator,<br>inference, persona,<br>modes |
| tools/                                       | 8.9k  | 30    | image engine, news,<br>etc.                              |
| memory/                                      | 7.0k  | 13    | SQLite/FTS5 + FAISS +<br>KG                              |
| perception/                                  | 6.8k  | 20    | vision, STT, TTS, OS<br>control                          |
| core/                                        | 6.6k  | 24    | paths, settings,<br>hardware profile                     |
| planning/                                    | 3.9k  | 24    | proactive daemon, task<br>planning                       |
| learning/                                    | 3.3k  | 12    | LoRA self-training                                       |
| plugins/                                     | 1.9k  | 28    | runtime plugin<br>manager                                |
| world/                                       | 1.5k  | 26    | world event bus, local<br>world bridge                   |
| integrations/,utils/,contracts/,system/,cli/ | 5.6k  | —     | misc                                                     |

**Four files carry ~1/3 of the codebase:** executor\_enhanced.py (15.0k LOC), engine.py (13.4k), gui/eli\_pro\_audio\_gui\_v2\_0.py (11.0k), router\_enhanced.py (7.1k). Next tier: labs\_tab.py (5.7k), memory.py (4.5k), deterministic\_grounding\_gate.py (4.3k).

## 3. Architecture, layer by layer

- **Boot / hardware** — core/hardware\_profile.py + core/startup\_hardware\_optimizer.py adapt n\_ctx / gpu\_layers / batch to whatever model + GPU are present (filename→ctx table; VRAM compute-buffer reservation). Model-agnostic.
- **Routing** — execution/router\_enhanced.py: regex-first with LLM-intent fallback + an explicit priority pipeline → one of **215 manifest capabilities (190 routable;** 180 executor SUPPORTED\_ACTIONS). Full reference with activation phrases: capabilities\_and\_actions.md.
- **Orchestration** — kernel/engine.py gates between the 12-stage cognition/orchestrator.py AgentOrchestrator (non-quick modes) and the parallel 14-agent cognition/agent\_bus.py AgentBus (quick

/ fallback). ReAct tool-chaining for non-CHAT actions. Selection now flows through one typed ExecutionPlan (execution/execution\_planner.py). See orchestration\_and\_agents.md for full detail.

- **Inference** — cognition/gguf\_inference.py: model-path resolved from env / settings (no baked model); RLock-serialized; live runtime override; hot-swap with vision models.
- **Memory** — memory/memory.py is the shared foundation (FTS5 + FAISS + KG via recall\_memory); two deliberate retrieval strategies sit on top (lightweight bus BusMemoryAgent vs heavyweight orchestrator OrchestratorMemoryAgent with HyDE + RAG + rerank). memory/knowledge\_graph.py, memory/vector\_store.py (FAISS, JSON meta).
- **Grounding / evidence (the crown jewel)** — runtime/deterministic\_grounding\_gate.py (4.3k), runtime/evidence\_ledger.py, runtime/evidence\_arbitration.py, runtime/control\_contracts.py, runtime/grounded\_remediation.py (1.6k), cognition/output\_governor.py. A layered, deterministic anti-confabulation system wrapped around the probabilistic model. Most local-LLM projects have nothing comparable.
- **Security** — runtime/security.py SecurityManager: **fail-closed shell gate** (ELI\_ALLOWED\_CMDS / the ELI Full Control toggle; unset = commands blocked), path allow-roots (ELI\_ALLOW\_ROOTS, defaults to project + home), app allowlist with a default-safe set, **SHA-256 custom-agent trust registry** (config/trusted\_agents.json), prompt-injection guard, SQL identifier validation.
- **Self-model** — cognition/reasoning\_modes.py: quick/fast/balanced all fastpath to quick, plus four private modes (chain\_of\_thought, self\_consistency, tree\_of\_thoughts, constitutional\_ai). Persona overlay, runtime/self\_improvement.py, LoRA self-training (learning/). Emergent internal-state surfacing (autonomy pressure, “anomaly room”) is intentional — see memory/eli-emergent-voice.

## 4. What’s genuinely strong

1. **The grounding / evidence layer.** Unusually rigorous; deterministic evidence gating around a probabilistic model is the right idea and well-developed. This is the part closest to genuinely frontier.
2. **Truly local + truly model-agnostic.** No call-home, hardware-adaptive, swappable models (verified — inference path carries no baked model identity).
3. **Real, fail-closed security.** Shell blocked by default, hash-gated custom agents, path/app allowlists. Most “AI agent” projects ship wide open.
4. **Breadth, integrated.** Vision, voice, OS control, memory, KG, plugins, self-training — all local, all wired into one pipeline.

## 5. Where it’s weak (grounded in the sweep)

1. **2,565 except Exception: blocks (+7 bare except:).** The dominant structural problem. Errors are swallowed into “skipped”/fallback/empty everywhere — which is precisely why bugs surface only via runtime logs. Failures are invisible by design. Frontier software makes failures **loud in dev, quiet in prod**; this swallows them uniformly. Also the main reason the system is hard to reason about. (Only 12 TODO/FIXME markers exist — not because the code is clean, but because failures are absorbed rather than flagged.)
2. **God-files.** Two ~13-14k-line modules (executor\_enhanced.py, engine.py) plus an 11k GUI. The executor is a giant if/elif action ladder. High regression surface, hard to hold in the head, painful to unit-test.
3. **Duplication & overlap.** The shadowed standalone image-engine module (1.75k LOC) was removed, leaving the single eli/tools/image\_engine/ package. runtime/ (80 files) has many near-duplicate personal\_memory\_\* / \*\_surface / \*\_response modules doing overlapping grounding work. Several plan representations coexisted (partly consolidated). Fingerprint of fast solo iteration — new code added beside the old, not folded in.
4. **Repo hygiene.** Committed junk at root: empty ... and [package-index-options] files, one-off patch\_gpu\_dynamic.py / patch\_sll\_bugs.py, three verify\_eli\_claims\*.sh versions, diag outputs, .coverage, and experimental/\*.zip binaries. Makes the repo look less serious than the code is.
5. **Tests are GREEN (measured 2026-07-01).** 205 test files; pytest tests/ = **7,348 passed /**

**0 failed / 45 skipped / 2 xfailed** (~8m16s on the .venv/GPU). The 5 former reds (deprecated smart\_home plugin, silent-swallow ratchet, stale blueprint ref) were all cleared 2026-07-03. The tests/claims/ contract layer makes it a real safety net.

6. **13 monkeypatch/globals() hacks** — mostly load-bearing (e.g. the CPU-clip vision fix), but they’re fragile seams worth tracking.

## 6. Honest verdict on “frontier, ground-breaking”

**In ambition and in specific subsystems — yes.** The grounding layer, the fully local model-agnostic design, and the integrated multi-modal local agent are genuinely ahead of most open local-assistant projects. The ideas are frontier-grade.

**In engineering discipline — not yet.** The 2,565 swallowed exceptions, the god-files, the duplication, and the clutter separate “an extraordinarily ambitious solo project” from “software others can build on.” None of that is a vision problem — it is consolidation and observability. The gap to “ground-breaking” is **subtraction and discipline, not more features.**

## 7. Highest-leverage work (the next month)

Ranked by effect-per-effort:

1. **Tame error-swallowing.** Replace blanket except Exception: pass/fallback with scoped exception types + a single structured error log (a ring buffer / table) you can actually watch. Keep the graceful degradation, but record every swallow. This alone makes the whole system debuggable and is the precondition for trusting any other change.
2. **Split the two god-files** along their natural seams (executor: action groups into per-domain modules behind the dispatch table; engine: pipeline stages into stage modules). Reduces regression surface and makes the pipeline readable.
3. **Delete duplication + clutter.** Collapse the overlapping runtime/ surfaces, remove root junk/one-off scripts. *(The suite is already green — 8,815 passing; this is now signal-to-noise hygiene, not a red-test cleanup.)*
4. **Consolidate the runtime/ surfaces.** The many personal\_memory\_\* / \*\_surface / \*\_response modules want to be a handful of well-named ones.

Do 1-3 and the engineering would match the ideas — which is the only thing standing between ELI and the label “frontier, ground-breaking software people can rely on.”